Neural Computing and Applications (2024) 36:14449–14464
https://doi.org/10.1007/s00521-024-09855-z

ORIGINAL ARTICLE

Software effort estimation using convolutional neural network and fuzzy clustering

Mohammad Azzeh[1] · Abedalrhman Alkhateeb[2] · Ali Bou Nassif[3]

Received: 3 May 2023 / Accepted: 12 April 2024 / Published online: 7 May 2024
© The Author(s), under exclusive licence to Springer-Verlag London Ltd., part of Springer Nature 2024, corrected publication 2024

Abstract

Adopting an efficient software process model is critical for building high-quality software applications. An important factor impacting the software development process is an accurate estimate of human effort required to complete the software project. While machine learning methods were historically used to develop estimation models, there has been little investigation into the potential of deep convolutional neural networks (DCNNs) for improving software effort estimation. One of the biggest obstacles in using DCNN for this purpose is the common nature of software datasets, which often consist of vectorized samples rather than matrices. To defeat this obstacle and reduce vagueness in software attribute measurement, this study uses Fuzzy theory to generate an appropriate two-dimensional datapoint representation. The fuzzy clustering is commonly used to split dataset samples into separate clusters, which can help to generate Fuzzy membership functions. This approach makes it easier to generate a two-dimensional array representation for each data sample based on the membership values, allowing it to be used as input to the DCNN model. The efficiency of the proposed model was thoroughly evaluated using PROMISE benchmark datasets. The findings based on mean absolute errors and standardized accuracy show that our proposed model produced very good performance with low error rates and outperformed several current state-of-the-art effort estimation models. Nonetheless, further research is needed to determine the impact of different cluster numbers and features on the performance of our model. In conclusion, this study emphasizes the possibility for incorporating DCNN into software effort estimates and highlights the viability of utilizing fuzzy modeling and clustering techniques to enhance the data representation of software datasets.

Keywords Effort estimation · Machine learning · Deep learning · Fuzzy logic

1 Introduction

Software development effort estimation encompasses the predictive process of determining the temporal, financial, and resource requisites for the execution of a software project [1–3]. Typically conducted at the project's outset, this estimation serves to provide project leaders with insights into the project's scope, enabling the establishment of pragmatic objectives and the efficient allocation of resources [4, 5]. The accurate estimation of software effort assumes paramount importance in ensuring the successful completion of a project [6, 7]. Inadequate estimations may lead to under-resourcing, impeding the project's ability to fulfill its objectives, while overestimations may result in unnecessary costs due to over-resourcing [8]. Consequently, the utilization of dependable estimation

✉ Mohammad Azzeh
m.azzeh@psut.edu.jo

Abedalrhman Alkhateeb
aalkhate@lakeheadu.ca

Ali Bou Nassif
anassif@sharjah.ac.ae

[1] Department of Data Science, Princess Sumaya University for Technology, Amman, Jordan

[2] Department of Computer Science, Lakehead University, Thunder Bay, Canada

[3] Department of Computer Engineering, University of Sharjah, Sharjah, UAE

methodologies and the continuous monitoring of project progress to facilitate necessary adjustments are imperative [9, 10].

The complex nature of software effort estimation presents several challenges, encompassing uncertainties, intricacies, and the expertise of software developers [3, 11, 12]. Contemporary software applications are characterized by complexity and interconnectivity, comprising numerous components and subsystems. This inherent complexity renders software development an uncertain process, susceptible to varied influences such as evolving requirements, emerging technologies, and unforeseen events, all of which can significantly impact project outcomes [13]. Consequently, the presence of uncertainty poses a daunting obstacle to the precise prediction of software effort. Typically, the degree of uncertainty in effort estimation is relatively high during the initial stages of software development, but it decreases as we advance to the final phases. The four basic sources of uncertainty in effort estimating are hypothesis errors, measurement mistakes, model errors, and estimator skill. To reduce imprecision in software measurement, approaches such as fuzzy logic, fuzzy set theory, and fuzzy clustering must be used.

The literature presents a spectrum of techniques and methodologies employed in the estimation of software effort, encompassing machine learning, algorithmic models, and expert judgment [14]. Notably, machine learning algorithms emerged as the predominant choice for constructing effort estimation models [15]. Despite the superior accuracy demonstrated by these models in comparison to algorithmic and statistical models, their application fell short of meeting industrial standards, primarily due to the inherent instability observed when operating across diverse environments and datasets [16]. Recently, deep learning algorithms have been used to solve many complex real-life problems. Nevertheless, deep convolutional neural network (DCNN) was not used to build effort estimation models because of the nature of software datasets. These datasets contain samples in the form of vectors that do not fit into DCNN model. The DCNN architecture is designed to accept two-dimensional (2D) data representations like images as input. Very few studies focused on the use of Deep Learning for predicting software effort from story points in agile software development [17–21]. Usually, the Agile stories are represented by text, which can be treated and handled using Recurrent Neural Network or DCNN if the story points content are vectorized. Nevertheless, these approaches need rich content in order to learn the context of story points well.

In light of the aforementioned challenges, we introduced a new software development effort prediction model based on integrating fuzzy model and "Deep Convolutional Neural Network" (DCNN). The fundamental premise underlying this approach involves the transformation of software dataset samples from data point space to 2D space through the utilization of fuzzy c-means (FCM), thereby enhancing the data compatibility with DCNN and reduce impression in software measurement. Central to this process is the application of FCM to partition the data points into a specified number of clusters where each sample could belong to multiple clusters but with different membership degrees. These membership values are processed and mapped to fuzzy functions along dimension of each feature, ensuring parity between the number of Fuzzy functions and number of clusters. Subsequently, a 2D array of membership degrees is generated for each data point by computing the membership degrees of individual values. This process is resulted in a matrix size of (m features $\times$ k *clusters*). The derived data is then employed as input for the constructed DCNN model, leveraging the capabilities of deep learning algorithms to discern intricate patterns inherent in the software data. To the end, this study marks the pioneering integration of fuzzy modeling and deep learning for software development effort estimation, representing a significant advancement in the field. The efficacy of our model is rigorously assessed against well-known effort prediction models using a comprehensive group of common software effort datasets.

This paper presents several key contributions, including:

(1) Introduction of a new feature engineering and data augmentation approach, employing FCM and Fuzzy modeling, to yield an enhanced data representation for software projects. This methodology effectively introduces substantial distribution to the representation of data points, thereby mitigating potential uncertainties arising from data measurement.
(2) Pioneering utilization of the CNN deep learning algorithm on tabulated data points, marking the first instance, to the best of our knowledge, of leveraging this approach to generate a reliable effort estimation characterized by commendable accuracy.

2 Background

2.1 Fuzzy logic

Fuzzy logic, as expounded in the literature [22], constitutes a mathematical framework tailored to address uncertainty and imprecision in information. Central to this approach is the fundamental premise that entities can possess partial truth or falsity, transcending the binary categorization of true or false. This unique characteristic endows fuzzy logic with the capacity to effectively manage vague or imprecise

data, a capability beyond the purview of traditional Boolean logic.

In the domain of fuzzy logic, the representation of a concept is facilitated through a membership function, which allocates a value ranging between zero and one to the concept, as depicted in Fig. 1. Notably, fuzzy logic serves as a potent tool for mitigating uncertainty through various avenues. For instance, it can be leveraged to make decisions in scenarios characterized by incomplete or ambiguous information, as documented in the literature [13]. Furthermore, its applicability extends to modeling intricate systems replete with numerous features and interactions. Based on that, Fuzzy logic offers the capability to capture the innate vagueness and imprecision within the system, thereby enabling more precise predictions and informed decision-making.

The construction of the fuzzy model entails the process of fuzzification, wherein each variable is transformed into a set of fuzzy functions. This transformation is achieved through two approaches: leveraging human expertise or utilizing automated tools. The latter being the methodology adopted in this research paper. Specifically, The FCM is employed to divide data points into clusters by generating cluster centroids alongside membership degrees [23]. The FCM can produce good cluster centroids by reducing intra-cluster variation while increasing inter-cluster distance. Since the data points can belong to multiple clusters with different membership values, the partition matrix (U) is constructed to store all membership degrees. The U matrix is iteratively updated while updating the cluster centroids. The iterative approach of centroids construction is repeated until the membership degrees of the data points converge, at which no more modifications are allowed.

2.2 Deep convolutional neural network

Deep convolutional neural networks (DCNNs) represent a pivotal neural network architecture widely applied in computer vision domains, encompassing tasks such as segmentation, object discovery, and image classification [24]. Characterized by an intricate structure comprising multiple convolutional, pooling, and fully connected layers, as depicted in Fig. 2, DCNNs are adept at extracting features from input matrices, including edges, corners, and textures, through the application of learnable filters (kernels) within the convolutional layers. These filters, typically of modest dimensions, traverse the input matrix to compute convolutions, yielding sets of feature maps that depict the presence of specific features across diverse locations within the input. The subsequent pooling layers serve to down sample the feature maps generated by the convolutional layers, effecting a reduction in the spatial dimensions of the feature maps to enhance model efficiency by diminishing the number of parameters. Common variants of pooling include max pooling, average pooling, and min pooling [25]. Furthermore, the fully connected layers, akin to traditional neural network dense layers, receive the output from the convolutional and pooling layers, flattening it into a one-dimensional vector, thereby subjecting it to a series of neurons to yield a final output. Additionally, the incorporation of Dropout layers serves to randomly deactivate certain neurons during training, mitigating overfitting and facilitating improved generalization to unseen data. The reason for using DCNNs in this situation is based on the two-dimensional data representation used for each data point, which, like images, allows the model to successfully capture the intricate patterns inherent in the software effort datasets.

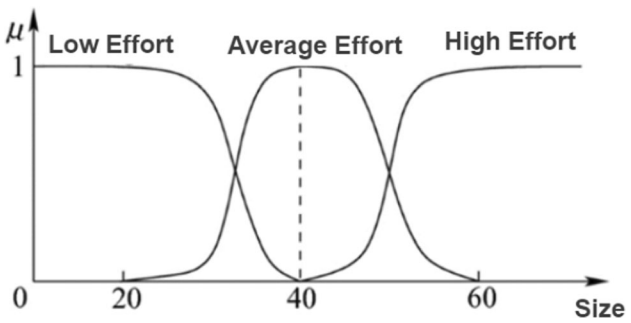


Fig. 1 Illustration of fuzzy sets with three membership functions for size variable

3 Related work

The domain of software effort estimation has been the subject of extensive exploration over the past four decades [2, 26], yielding a substantial corpus of research papers dedicated to addressing diverse challenges, primarily centered on enhancing estimation accuracy. Critical to software effort estimation is the prerequisite for robust and dependable measurement of software attributes. These features are categorized into two distinct groups: internal and external features. Internal features encompass quantifiable characteristics inherent to software projects, such as project size and complexity, while external features encapsulate the impact of product behavior and environmental factors on software development, including team communication, reliability, and multisite development. Notably, among these features, project size assumes paramount significance as the principal driver of effort, contributing significantly to the credibility of software effort estimation and serving as a mitigating factor for software project risks, surpassing the influence of other internal or external features.

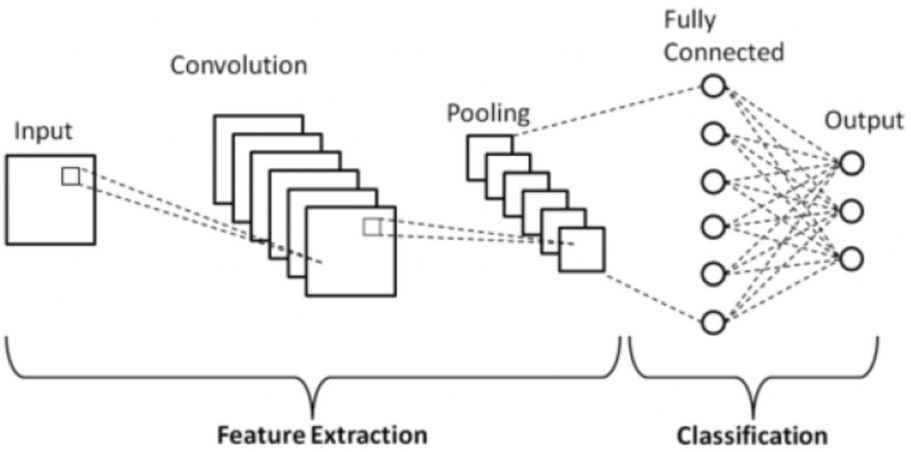


Fig. 2 Illustration of deep convolutional neural network

Throughout the literature, two basic software effort estimation methodologies have been identified [1, 27]: model-based and expert opinion [14]. Expert-based methodologies rely on estimations derived from similar experienced scenarios. Model-based techniques, on the other hand, use algorithms rather than expert judgment to establish the link between input attributes and effort [28, 29]. The latter technique is divided into two parts: the parametric approach and the learning approach, which uses machine learning algorithms. Machine learning-based techniques entail training models on historical data to prognosticate effort for new projects. These techniques encompass regression analysis, decision trees, neural networks, and support vector machines. Notably, these algorithms demonstrate commendable performance when confronted with datasets characterized by intricate structures and a large number of samples, exhibiting promising potential in enhancing the accuracy of software effort estimation, particularly in scenarios where a substantial volume of historical data is accessible [30, 31]. However, these techniques require extensive data preprocessing, feature selection, and model tuning to achieve accurate estimations. Several studies have compared the accuracy of different software effort estimation techniques [7, 14, 16, 32, 33]. They found that machine learning-based techniques outperform algorithmic and expert judgment-based approaches in terms of accuracy. While other studies have found that algorithmic models perform better in some scenarios. Some studies have also investigated the impact of project factors such as project size, team experience, and development process on effort estimation accuracy.

This list is hardly complete. The prior works [5, 34–37] have comprehensively surveyed the spectrum of studies that construct estimation models utilizing machine learning methods in the existing literature. Notably, estimation by analogy emerged as the most extensively investigated method in the literature [10, 16, 28, 38]. However, this model encompasses a multitude of configurable parameters necessitating meticulous setup, including optimal feature selection, determination of their weights, selection of the best k-nearest neighbors, and identification of an appropriate adaptation technique. Nevertheless, these algorithms operate on a global learning paradigm, potentially unsuitable for software datasets that necessitate local data for the development of more robust estimation models. Locality-based methods leverage akin training projects to estimate new projects during testing [1, 39], encompassing feature selection, classification, and clustering algorithms. Several studies have delved into the integration of locality within their prediction models, affirming its potential to augment accuracy. Gallego et al. [40], Minku et al. [39], and Azzeh et al. [41] have explored diverse locality-based approaches for effort estimation, concurring on its capacity to enhance accuracy. Conversely, certain studies have underscored factors influencing estimation accuracy, such as within and cross-project estimations, leading to the development of suboptimal models. For instance, some studies have expanded "within-company" training sets by incorporating "cross-company" instances to construct machine learning models for predicting "within-company" contexts [3]. Notably, models trained on "within-company" instances have demonstrated superior or comparable predictive performance relative to the aforementioned models. Additionally, numerous studies have applied neural networks integrated with fuzzy logic, as evidenced by [33, 42–45].

Studies on using deep learning for effort estimation are quite rare and mainly focus on predicting effort from story points using natural language processing algorithms [17–21]. Story points are an alternative measure of effort in agile software development. The input of these models is usually text of stories in which the model attempt to learn the contextual embedding of these stories and then builds text representations for them [17–19]. Later, these representations are linked with effort through a fully connected

neural network model. Morais [17] used natural language processing to predict story points from 16 open-source projects. Phan and Jannesari [18] used a graph neural network with TF-IDF text representation to predict story points from user stories. Kassem et al. [19] applied Hierarchical attention networks with two attention mechanisms based on words and sentences. Marapelli et al. [20] applied a recurrent neural network with CNN to extract features from user stories and then incorporated sequence learning to predict the correct number of story points.

However, the DCNN was not used before for the problem of effort estimation because of the input mismatch. The DCNN requires input in the form of 2D matrix, which is not available on the current datasets. Therefore, we decided to make automatic feature engineering for these datasets to build a matrix representation for each data point.

4 The proposed model

This section explains the framework of the proposed model, as illustrated in Fig. 3. There are two main phases: data representation and prediction model construction. The first phase accepts dataset that contains data points and produces 2D data representation for each data point using the proposed fuzzy model. The second phase accepts the generated 2D data point representation as input to the constructed DCNN model. This model is primarily designed to detect and incorporate the complicated patterns inherent in the newly created data structure.

4.1 Feature engineering task

The initial dataset, characterized by n data points and m attributes, is subjected to the FCM algorithm, aiming to isolate samples into distinct clusters contingent upon the predetermined quantity, k, of clusters. Each cluster centroid is delineated by m values. The selection of the k value exerts substantial influence on the configuration of the resultant data points. Hence, in order to ascertain the best number k value that most suitably align with the distribution of the dataset, the Xie Beni (XB) clustering validity index, as illustrated in Eq. 1, is consulted [46].

$$\mathrm{XB} = \frac{\sum_{j=1}^{k}\sum_{i=1}^{n} U_{ij}^{2}\left\|C_j - x_i\right\|}{n \times \min_{j,t}\left\|C_j - C_t\right\|} \tag{1}$$

where C_j is the jth center vector, U is the partition matrix, x_i is the ith observation, and $\|\|$ is Euclidean distance.

The fuzzy C-means technique produces outputs that include the partition matrix (U) and cluster centroids. Equation 3 depicts the partition matrix, which delineates the membership values of each datapoint inside each cluster. The centroids of the clusters and their accompanying membership values are the basic building blocks for the membership functions. The cluster centroid (C_r) is explained as a vector with m values, as seen in Eq. 2, wherein each value in this vector (i.e., c_{rj}) signifies the projection of the centroid of the *rth* cluster along the *jth* dimension. Equation 4 provides a comprehensive representation of the matrix of cluster centroids.

$$C_r = [c_{r1} \quad c_{r2} \ldots \quad c_{rm}]r \in \{1, 2 \ldots k\} \tag{2}$$

where k is the number of clusters.

$$\begin{bmatrix} \mu_{11} & \cdots & \mu_{1k} \\ \vdots & \ddots & \vdots \\ \mu_{n1} & \cdots & \mu_{nk} \end{bmatrix} \tag{3}$$

$$\begin{bmatrix} c_{11} & \cdots & c_{1m} \\ \vdots & \ddots & \vdots \\ c_{k1} & \cdots & c_{km} \end{bmatrix} \tag{4}$$

where each μ_{ir} represents the membership degree of *ith* data point to the *rth* cluster.

"The resulting centroids and partition matrix U were used to generate fuzzy functions of membership for each feature in the Fuzzy Model. Each fuzzy function describes the distribution of data points inside a certain cluster. As a result, the fuzzy membership function has to be convex in order to accurately express the degree of link between proximal values and the function's center. Figure 4 illustrates how the plot of the Gaussian fuzzy membership function is utilized to construct the fuzzy model. The Gaussian function requires two parameters: (1) center and (2) width. Each fuzzy function's center is generated using the centroid vector of the relevant cluster, and the breadth is produced by examining the total distance between data points and their membership Along each dimension, as shown in Eq. 5. The use of the logarithm of the

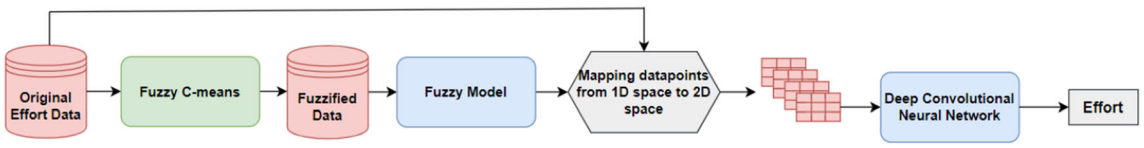


Fig. 3 Framework of the proposed model

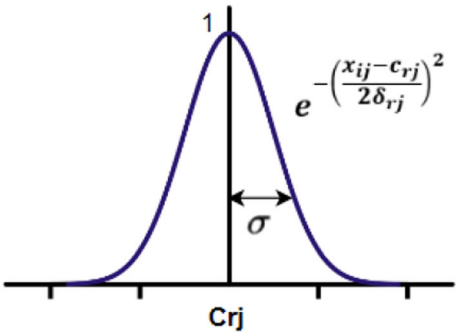


Fig. 4 The Gaussian membership functions

membership degree helps decrease the influence of data points far from the center. Specifically, for each feature, the centroids (i.e., c_{1j}, c_{2j}, … c_{rj}…c_{kj}) are selected to calculate the width of fuzzy membership functions of that feature. The outcome of this process is the fuzzy model with multiple membership functions as depicted in Fig. 7. We can see there are three features and three clusters in each feature, where each cluster is represented by a Gaussian membership function across each feature".

$$\delta_{rj} = \frac{1}{n}\sum_{i=1}^{n}\sqrt{\frac{-\left(x_{ij}-c_{rj}\right)^2}{2\times\log(U_{ir})}} \tag{5}$$

$$e^{-\left(\frac{x_{ij}-c_{rj}}{2\delta_{rj}}\right)^2} \tag{6}$$

where x_{ij} is the *ith* data point at *jth* feature. C_{rj} is the value of *jth* index in the *rth* centroid. U_{ir} is the membership degree of *ith* datapoint in the *rth* cluster. δ_{rj} is the standard deviation of data point that belong to the *jth* cluster (Fig. 5).

After constructing the membership functions, the new 2D data representation for each sample is created. To accomplish that, we should find membership degrees that match each datapoint in the universe of discourse of each feature. This involves computing membership degrees for each input value across all dimensions. This method produces a matrix of membership values, as given in Eq. 7, which effectively represents how closely the data point vector conforms to all clusters across all dimensions. Equation 7 shows the matrix representation of a single data point, including membership degrees across all features and cluster centroids.

$$\begin{matrix}\mu_{11} & \mu_{12} & \mu_{13}\\ \mu_{21} & \mu_{22} & \mu_{23}\\ \mu_{31} & \mu_{32} & \mu_{33}\end{matrix} \tag{7}$$

4.2 Deep convolutional neural network construction

This section describes the construction of our proposed DCNN model as shown in Fig. 6. The construction of a Deep Convolutional Neural Network (DCNN) entails the sequential arrangement of multiple layers, including convolutional, pooling, and fully connected layers, in conjunction with activation functions and regularization techniques. This amalgamation facilitates the development of a robust deep neural network adept at discerning intricate features from raw input data. The DCNN model receives the generated matrices from the fuzzy model as input and endeavors to predict the output variable. Illustrated in Fig. 6, the DCNN is composed of two 2D convolutional layers, two max pooling layers, and a fully connected layer. Each convolution layer is comprised of 16 kernels (i.e., filters) with a size of (3,3), and is followed by Relu layers. This number of kernels has been chosen after searching for the best number that produces better performance among all datasets. The stride size in convolution and pooling layers is set as (1,1). The fully connected layers contain Flatten layer and two hidden layers with 32 and 16 neurons each. The activation function for both hidden layers is Relu function. A dropout layer with probability 0.2 has been added between hidden layers. The output layer contains only one neuron with Relu activation function The DCNN model is trained by utilizing Adam optimizer and root-mean-squared errors as loss function.

5 Research methodology

5.1 Datasets

To experimentally evaluate the proposed model and the comparator models, a selection of well-known benchmark datasets from the PROMISE repository [47] were used. These datasets ("Desharnais, Kemerer, Albrecht, Maxwell, China, Telecom, and Nasa") are distinguished by a unique collection of attributes divided into four categories: size features, software development features, environment factors, and general project data. In these datasets, the output variable represents effort, which is measured in person-hours or person-months. Descriptive statistics of the utilized datasets are presented in Table 1, and the complete dataset description is available in Appendix A. The datasets exhibit variability in size, contingent upon the number of projects and variables encompassed within each dataset. Larger datasets tend to be more representative of the population and can improve the learnability of DCNN model. Also, these datasets typically contain a mix of

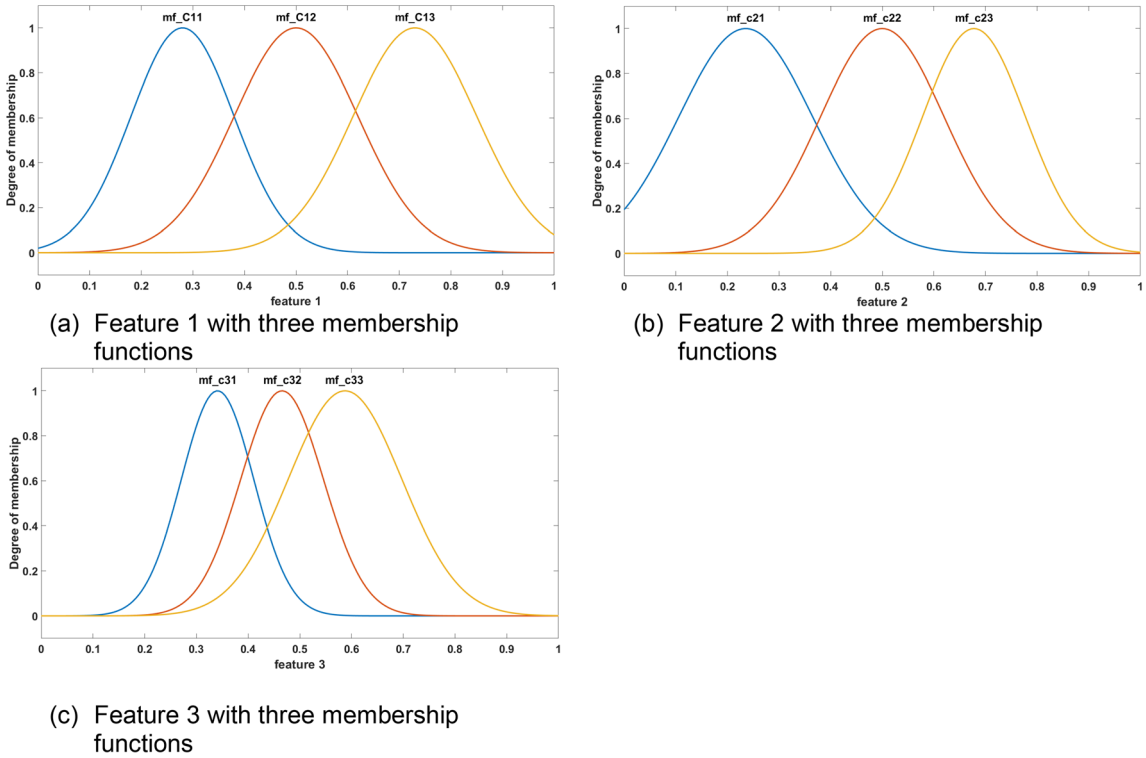


Fig. 5 The constructed membership functions over three features. We can notice that the three clusters are projected over three features. The center and width of these membership functions are computed from partition matrix and centroid matrix

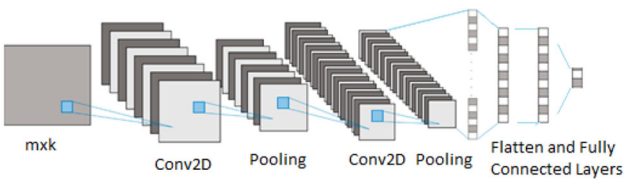


Fig. 6 DCNN model architecture

quantitative and categorical variables. Quantitative variables include project size, team size, and lines of code, while categorical variables include project type, programming language, and development methodology. Therefore, we focus on numeric features only because they are eligible for clustering and fuzzification. However, Software effort datasets may be heterogeneous, meaning they contain projects with different characteristics, such as project size, team size, and programming language [41, 48]. Heterogeneous datasets can be more representative of the population but can also make it challenging to build accurate estimation models.

Overall, the characteristics of software effort datasets can impact the accuracy of the estimation models. It is essential to consider the variables included in the dataset carefully, the presence of missing data and outliers, and the potential correlation and heterogeneity of the data to build accurate and robust estimation models. Therefore, before using these datasets, we preprocessed them to be clean and ready to use without any potential bias. Few datasets have a very small number of missing values, thus, we decided to remove these records as this is the best practice used in

Table 1 Statistical properties of the employed dataset

Dataset	Feature	Size	Effort data					
			Unit	Min	Max	Mean	Median	Skew
Albrecht	7	24	Months	1	105	22	12	2.2
ISBSG	10	505	Hours	668	14,938	2828	1634	2.1
Desharnais	12	77	Hours	546	23,940	5046	3647	2.0
COCOMO	17	63	Months	6	11,400	683	98	4.4
China	18	499	Hours	26	54,620	3921	1829	3.92
Maxwell	27	62	Hours	583	63,694	8223	5189	3.26

Table 2 Configuration parameters

Machine learning algorithm	Configuration parameters
SVR	Kernel type = ['radial basis function', 'linear']
	Penalty parameter (C) = [0.05,0.1,0.3,0.5]
	Kernel coefficient (gamma) = [0.1,0.3,0.5]
ABE	Number of nearest neighbors k = [1, 3, 5, 7]
	Adaptation function = ['weighted distance adaptation', 'null']
RF	Number of estimators = [50,100,150,200]
XGBoost	Number of estimators = [50,100,150,200]
MLP	Number of hidden layers [2–4]
	Number of neurons in the hidden layers = [5, 10, 15]
	Activation function = ['relu', 'sigmoid']

Table 3 MAE results

Dataset	F-DCNN	ABE	SVR	RF	XGB	MLP
Albrecht	**1.1**	6.3	4.4	4.3	3.5	6.1
	(0.29, 1.85)	(3.57, 9.02)	(2.614, 6.262)	(0.54, 7.99)	(0.34, 6.68)	(3.84, 8.34)
Desharnais	**1044**	1925	1316	1248	1066	1850
	(516, 157)	(1386, 2466)	(879, 1753)	(835, 1661)	(535, 1597)	(1329, 2372)
China	**899**	1165	995	1002	916	1605
	(650, 1149)	(913, 1418)	(682, 1309)	(793, 1211)	(665, 1167)	(1295, 1916)
Maxwell	**1749**	2252	4397	1751	1846	5109
	(780, 2717)	(1188, 3316)	(2946, 5848)	(779, 2724)	(1149, 2545)	(3004, 7215)
COCOMO	**160**	549	271	178	174	540
	(1, 321)	(166, 932)	(89.4, 453.2)	(7.8, 348.2)	(4.0, 344.4)	(164, 916)
ISBSG	**606**	1672	1586	742	703	1876
	(505.5, 706.7)	(1464, 1880)	(1428.0, 1744.9)	(624.8, 859.3)	(588.4, 818.8)	(1649, 2103)

software effort estimation. For example, the Desharnais dataset contains four missing data points out of 81 records, so deleting these projects is the common solution for these data points [27, 48]. We also checked the datasets against outliers that are significantly different from the rest of the data points. We converted data points into z scores to see how many standard deviations away the data points are from the mean. Fortunately, we did not find significant outliers in these datasets.

5.2 Evaluation measures

Although there are many evaluation metrics which can be used to assess the accuracy of effort estimation models, we favored to us the most unbiased and reliable evaluation metrics. Mainly, we used three robust evaluation measure that have ability to cover multiple aspect of the model accuracy. First, we used mean absolute errors (MAEs) which can measure the average of differences between the estimated ($\widehat{y}_i$) and actual effort (y_i) as shown in Eq. 8. The second metric is the Standardized Accuracy (SA), as shown in Eq. 9, which indicates the percentage of accuracy

Table 4 SA and effect size results

Dataset		F-DCNN	ABE	SVR	RF	XGB	**MLP**
Albrecht	SA	**1.0**	0.7	0.8	0.8	0.8	0.7
	Δ	**0.7**	0.5	0.6	0.6	0.6	0.5
Desharnais	SA	**0.8**	0.6	0.7	0.7	**0.8**	0.6
	Δ	**0.8**	0.6	0.7	0.7	**0.8**	0.6
China	SA	**0.8**	0.7	0.7	0.7	**0.8**	0.6
	Δ	**0.5**	0.4	0.4	0.4	0.4	0.3
Maxwell	SA	**0.8**	0.6	0.3	0.7	0.7	0.1
	Δ	**0.5**	0.3	0.1	0.4	0.4	0.1
COCOMO	SA	**0.9**	0.5	0.7	0.8	0.8	0.5
	Δ	**0.5**	0.2	0.3	0.4	0.4	0.2
ISBSG	SA	**0.9**	0.8	0.8	**0.9**	**0.9**	0.8
	Δ	**2.9**	2.5	2.5	2.8	2.8	2.4

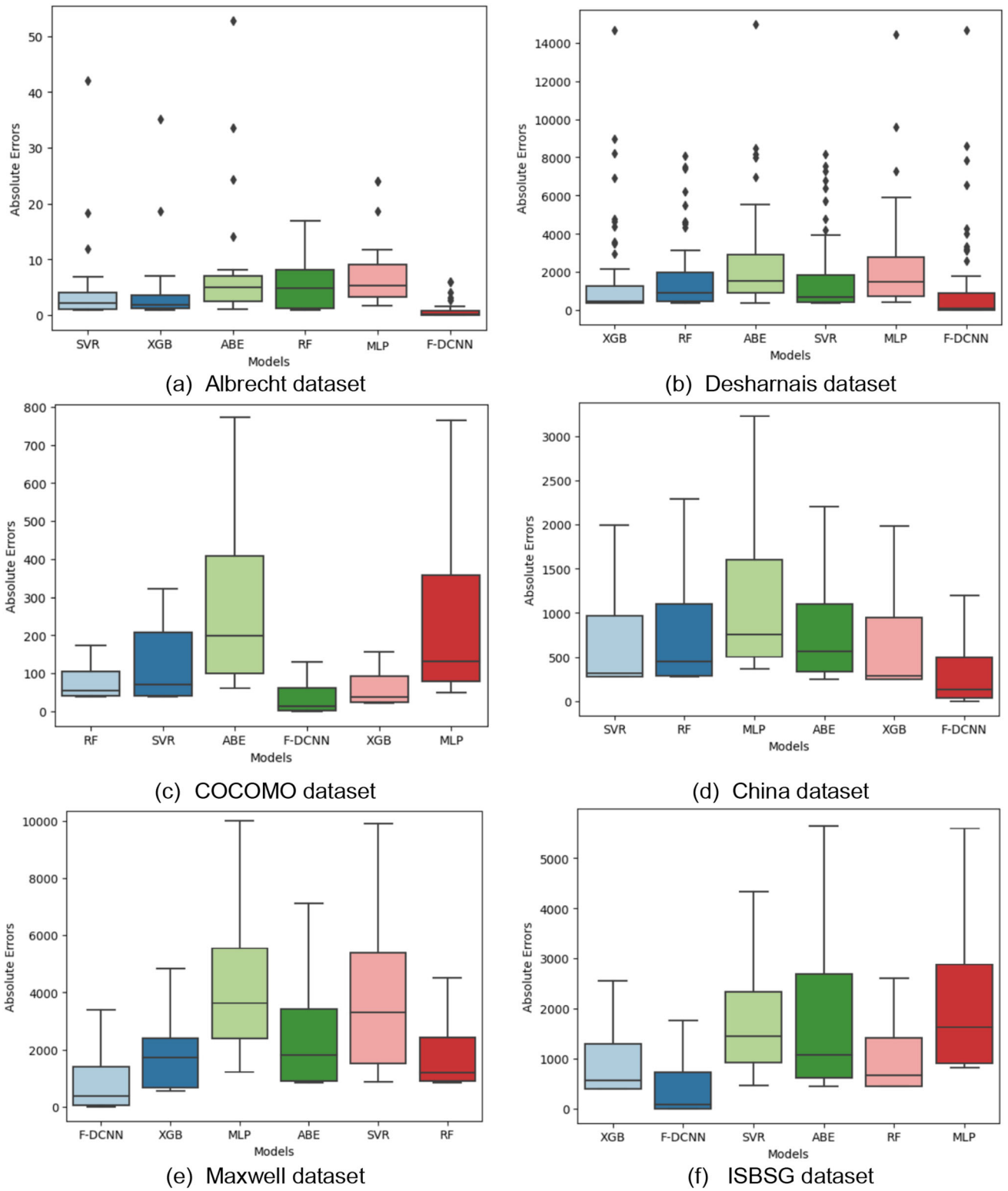


Fig. 7 Boxplots of absolute errors for all models

improvement over random guessing. Finally, the effect size (Δ) is used to appraise the disparity between actual and estimated effort as shown in Eq. 10. Although standardized accuracy and effect size give useful insights into the accuracy of effort estimates, it is critical to understand that these metrics are not complete and that other measures may

Table 5 Statistical significance test (p-value) between F-DCNN and other models based on residuals

Dataset	ABE	SVR	RF	XGB	MLP
Albrecht	**< 0.01**	**< 0.01**	**0.04**	**0.04**	**< 0.01**
ISBSG	**< 0.01**	**< 0.01**	0.23	0.21	**< 0.01**
Desharnais	**< 0.01**	**< 0.01**	**0.03**	0.32	**< 0.01**
COCOMO	**< 0.01**	**0.04**	0.32	0.45	**< 0.01**
China	**0.02**	0.26	0.35	0.52	**< 0.01**
Maxwell	**0.03**	**< 0.01**	0.28	**0.04**	**< 0.01**

be more appropriate in certain settings. Furthermore, it is critical to carefully analyze the constraints of each assessment measure and to use numerous metrics in order to acquire a thorough picture of effort estimation accuracy.

$$\mathrm{MAE} = \frac{\sum_i^n |y_i - \hat{y}_i|}{n} \tag{8}$$

$$\mathrm{SA} = 1 - \frac{\mathrm{MAE}}{\overline{\mathrm{MAE}}} \tag{9}$$

$$\Delta = \frac{\mathrm{MAE} - \overline{\mathrm{MAE}}}{\overline{\mathrm{SD}}} \tag{10}$$

where $\overline{\mathrm{MAE}}$ and $\overline{\mathrm{SD}}$ are mean absolute errors and standard deviations of errors for random guessing models.

5.3 Experimental setup

The Leave-One-Out Cross-Validation (LOOCV) method was used to validate the suggested model [49]. This approach works by using each data point in the dataset as a testing set, with the remaining data points serving as the training dataset. The model is then trained on the training set and assessed on the test data point, with the evaluation measure (e.g., MAE, SA, and effect size) recorded for each occurrence. This iterative procedure is performed n times until each data point is used as a testing set, after which the average assessment metrics across all data points are calculated. LOOCV provides an unbiased measure of the model's performance by using all available data for training and testing. However, it may be computationally intensive, especially for large datasets, because it requires training and testing the model for each occurrence in the dataset [49]. LOOCV is widely used with small datasets and basic models since it offers a more accurate estimate of the model's performance. Nonetheless, for bigger datasets, k-Fold cross-validation with a lower value of k is frequently preferred since it may achieve a compromise between bias and variance while being less computationally costly.

The suggested model was contrasted with a variety of benchmark effort prediction models based on various machine learning methods: Analogy Effort Estimation (ABE), "Support Vector Regression (SVR), Multiple Linear Regression (MLR), Random Forest (RF), Extreme

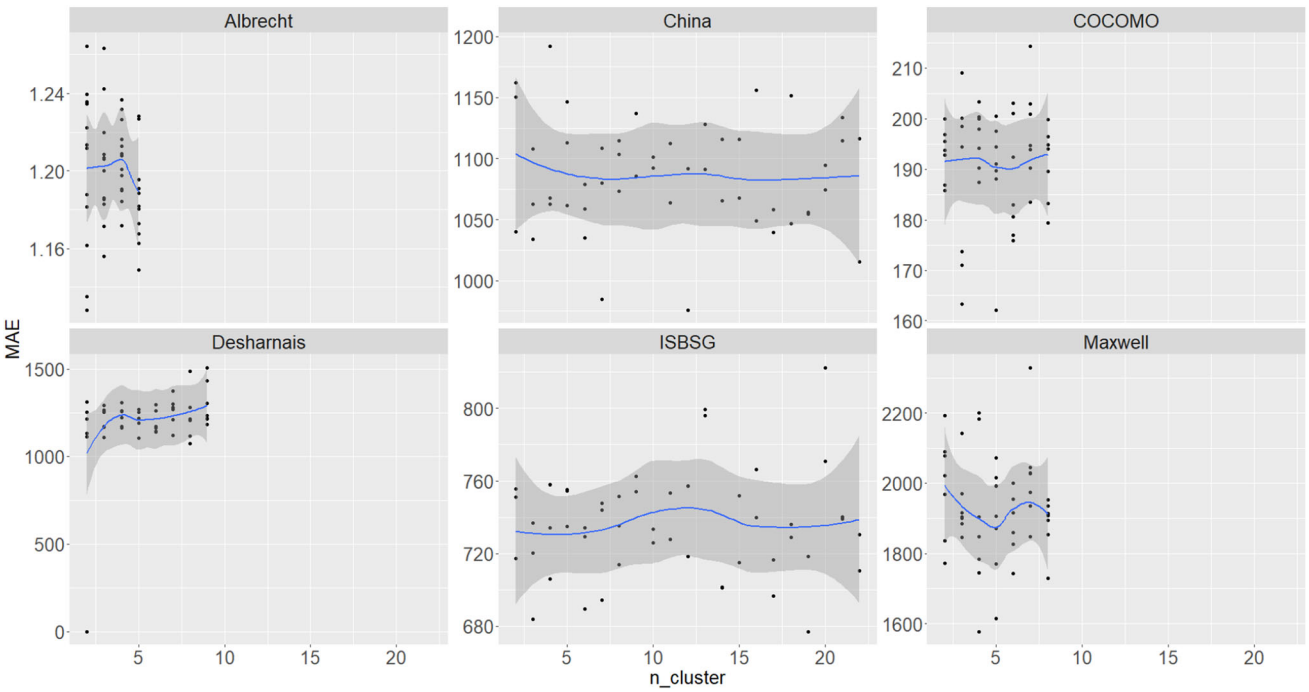


Fig. 8 Selectivity analyses of uncertainty

Gradient Boosting (XGBoost), and Multi-Layer Perceptron Neural Network (MLP)". Table 2 lists the setup parameters customized for each baseline technique. The best settings for these factors are determined by the quality of datapoints and the specific situation at hand. Parameter tuning comprises experimenting with various value combinations, resulting in the selection of the combination that produces the best performance on a validation set or through cross validation. As a result, a grid search technique was used to fine-tune these parameters for each dataset, with fivefold cross-validation.

6 Results

This section gives a full summary and analysis of the findings obtained using our suggested model and other comparison models. To achieve fair comparisons, all models were trained and tested on identical data sets partitioned using Leave-One-Out Cross-Validation (LOOCV). Table 3 shows the Mean Absolute Error (MAE) findings. The results of each dataset are displayed in two rows: the upper row presents the mean MAE, while the lower row presents the confidence interval of absolute residuals for each model. A lower MAE indicates higher model performance. Significant variability in MAE values were discovered between datasets, which were attributed to the different units employed to quantify the effort variable in each dataset. As a result, model comparisons were undertaken along each dataset. The MAE results consistently show that the proposed F-DCNN model outperforms other models, delivering the lowest MAE across all datasets and confirming our approach's effectiveness. However, it is worth noting that the second-best performing model, XGB, has MAE values that are almost identical to those of our proposed model across all datasets. When we did the Wilcoxon signed rank test to compare our model's absolute errors to those of XGB, we discovered a substantial discrepancy, raising concerns about our model performance. One plausible factor contributing to this disparity could be the selection of the number of clusters, which evidently exercised a substantial impact on the creation of data representation. While the XB clustering validity index was employed to ascertain the optimal number of clusters, the implications of this choice warrant further investigation, we believe that we need to tune this factor through optimization process. Another reason is that we use membership values of instead of the actual value which might reduce the interpretability of the constructed model.

On the contrary, the level of improvement demonstrated by each model was compared to random guessing to determine its predictive characteristics. To this goal, The SA and effect size were used, as shown in Table 4. To conserve space, the SA and Δ findings for each dataset were given in two rows. A higher SA value suggests better model performance. A model with a SA of 0.9 represents 80% improvement over the random guessing model. On the other hand, the effect size is an additional statistic for assessing the confidence of accuracy increases. A model with an effect size greater than 0.5 is considered credible. The suggested model regularly showed improvement percentages ranging from 0.8 to 1.0, with impact sizes of ≥ 0.5. Furthermore, the MLP performed somewhat better than the COCOMO and Maxwell datasets. Similarly, ABE and SVR outperformed the COCOMO and Maxwell datasets, respectively.

To confirm the results, the boxplot of absolute residuals for all models grouped by datasets is drawn in Fig. 7. Overall, it can be noticed that box length of F-DCNN model is much smaller than other models which shows less variability in absolute errors. For some datasets, there are overlaps between first and third quartiles of F-DCNN and those of XGB and RF, which demonstrate insignificant differences.

Above all, we also did statistical significance tests using Wilcoxon sum rank test over all datasets to examine the statistical difference between FDCNN and other effort estimation models. The Wilcoxon sum rank test has been used because the distributions of absolute errors are not normally distributed based on Kolmogorov–Smirnov test. The results are presented in Table 5, where each column represents the compared models with FDCNN and cells contain the resulted p-values. We can see that the FDCNN produces significant different predictions than ABE and MLP over all datasets. Likewise, we found that our model behaves differently from SVR over all datasets except over China dataset. On the other hand, we cannot find any significant difference between predictions generated by FDCNN and RF over large and medium datasets such as ISBSG, Maxwell, and China. The same pattern can be found when compare FDCNN with XGB where we only can find a significant difference over Albrecht and Maxwell datasets.

We also conducted a sensitivity analysis experiment to study how the number of clusters dramatically affects the stability and accuracy of the F-DCNN model under different degrees of uncertainty. We changed the number of clusters from 2 to the square root of number of samples ($\sqrt{n}$), without altering any other parameters. To ensure consistency, we repeated the experiments for small datasets to have the same number of runs as large datasets. We made 50 runs for each dataset that used all available cluster numbers. We then ran the F-DCNN model on all datasets and recorded accuracy values. The analysis results are illustrated in Fig. 8, where each subplot show sensitivity analysis for a specific dataset. The x-axis displays the

number of clusters whereas the y-axis shows MAE. The Lowes smoothing with confidence interval was used to show the relationship between number of clusters and MAE. The gray area denotes the confidence interval of the samples. However, our results indicate that the F-DCNN model behaves quite stable under different degrees of uncertainty, as majority of MAE values fall within the confidence interval across all datasets.

7 Threats to validity

There exist four distinct categories of threats to validity: internal, external, construction, and conclusion. To assess internal validity, three assessment metrics (MAE, SA, and effect size) were employed to validate the schema. These measures were selected for their impartiality and widespread utilization in effort prediction studies. Conversely, measures such as mean magnitude relative errors (MMREs) and median magnitude relative errors (MdMREs) were deemed biased and asymmetrical, thus unsuitable for evaluating effort estimation models. Regarding external validity, the study focused on six datasets commonly utilized in effort estimation research, omitting others due to inadequate size and feature coverage for DCNN fitting, leading to the exclusion of datasets such as Telecom and Nasa.

Construct validity was centered on the selection of machine learning algorithms and their parameters. The XB validity index was utilized to control the finest number of clusters for each dataset within our framework. Subsequently, the DCNN model was tailored to correspond to the data complexity. As for the comparative machine learning algorithms, the most frequently employed algorithms with finely tuned configurations for each kernel were employed. Lastly, conclusion validity primarily pertained to the statistical analysis methods employed in the study. The Wilcoxon signed rank test and effect size were utilized to discern significant differences among the effort estimation models. Furthermore, the choice of validation procedure was recognized as influential, prompting the adoption of LOOCV as recommended in a study by Kocaguneli and Menzies [49] to mitigate biases associated with k-folds in data point selection during training and testing.

8 Conclusion

This study presents a novel effort estimate model based on the merging of a fuzzy model and a Deep Convolutional Neural Network. The fuzzy model is used to understand the underlying data structure and create a matrix representation for each data point in the dataset, based on fuzzy clustering concepts. Furthermore, a unique feature engineering technique is provided, allowing for the generation of useful data point representations that can be integrated with deep learning models. Following that, a deep CNN model is developed to accommodate such data. When compared to typical machine learning methods, the F-DCNN model outperforms them in terms of detecting subtle patterns in software datasets. To assess its efficacy, the proposed model is compared to a collection of baseline models across different benchmark datasets, providing encouraging findings that exceed the vast majority of previous machine learning models. In addition, a sensitivity analysis is performed to investigate the effect of increasing the degree of uncertainty, controlled by the number of clusters, on the model's accuracy, demonstrating its overall stability. Nonetheless, further research is needed to understand the influence of feature selection, dimensionality reduction, and DCNN setup factors on the accuracy of effort estimation. Finally, the managerial implication of our study can assist managers and team leaders to better utilize and optimize human resources allocation by reflecting the predicted effort estimate on their plans. By minimizing level of uncertainty, the manager can make decision with high confidence and less risk. The proposed model can be updated and improved over time as more data becomes available. This continuous learning process can result in increasingly accurate effort estimations. In conclusion, while there are several benefits to using the proposed model for software effort estimation, managers should be aware of the challenges and actively manage the implementation process to maximize the advantages of deep learning and fuzzy modeling.

Appendix A

Complete dataset descriptions

See Tables 6, 7, 8, 9, 10 and 11.

Table 6 Albrecht dataset description

Feature	Description	Type
KLOC	Line of code	Numeric
AFP	Adjusted function points	Numeric
INC	Count of input functions	Numeric
OC	Count of output functions	Numeric
INQ	Count of query functions	Numeric
FC	Count of file processing	Numeric

Table 7 Kemerer dataset description

Feature	Description	Type
KSLOC	Kilo line of code	Numeric
AdiFP	Adjusted function points	Numeric
RawFP	Unadjusted function points	Numeric
Duration	Duration of project	Numeric
Language	Programming language	Nominal
Hardware	Hardware resources	Nominal

Table 8 Desharnais dataset description

Feature	Description	Type
ExpEquip	Experience of equipment	Numeric
ExpProjMan	ProjectManger experience	Numeric
Transaction	Number of transactions	Numeric
Entities	Number of entities	Numeric
RawFP	Raw function points	Numeric
AdjFactor	Adjustment factor	Nominal
AdjFPs	Adjusted function points	Numeric
Dev.Env	Development environment	Numeric

Table 9 COCOMO dataset description

Feature	Description	Type
DEV	Development mode	Categorical
ACAP	Analysts capability	Numeric
PCAP	Programmers capability	Numeric
AEXP	Application experience	Numeric
MODP	Modern programming practices	Numeric
TOOL	Use of software tools	Numeric
VEXP	Virtual machine experience	Numeric
LEXP	Language experience	Numeric
SCED	Schedule constraint	Numeric
STOR	Main memory constraint	Numeric
DATA	Data base size	Numeric
TIME	Time constraint for CPU	Numeric
TURN	Turnaround time	Numeric
VIRT	Machine volatility	Numeric
CPLX	Process complexity	Numeric
RELY	Required software reliability	Numeric
LOC	Line of code	Numeric

Table 10 Maxwell dataset description

Feature	Description	Type
Syear	Start year of the project	Numeric
App	Application type	Nominal
Har	Hardware platform	Nominal
Dba	Database	Nominal
Ifc	User interface	Nominal
Source	Where developed	Nominal
Telonuse	Telon use	Nominal
Nlan	Number of different development languages used	ordinal
T01	Customer participation	ordinal
T02	Development environment adequacy	ordinal
T03	Staff availability	ordinal
T04	Standards use	ordinal
T05	Methods use	ordinal
T06	Tools use	ordinal
T07	Software logical complexity	ordinal
T08	Requirements volatility	ordinal
T09	Quality requirements	ordinal
T10	Efficiency requirements	ordinal
T11	Installation requirements	ordinal
T12	Staff analysis skills	ordinal
T13	Staff application knowledge	ordinal
T14	Staff tool skills	ordinal
T15	Staff team skills	ordinal
Duration	Duration of project from specification until delivery, measured in months	Numeric
Size	Application size function points measured using the experience method	Numeric
Time	Time = Syear − 1985 + 1, with evels: 1, 2, 3, 4, 5, 6, 7, 8, 9	Numeric

Table 11 China dataset description

Feature	Description	Type
AFP	Adjusted function points	Numeric
Input	Input count	Numeric
Output	Output count	Numeric
Enquiry	Enquiry count	Numeric
File	File count	Numeric
Interface	Interface count	Numeric
Added	Added functions count	Numeric
Changed	Changed functions count	Numeric
Deleted	Deleted function count	Numeric
PDR AFP	–	Numeric
PDR UFP	–	Numeric
NPDR AFP	–	Numeric
NPDU UFP	–	Numeric
Resource	Resource level	Numeric
Dev.Type	Development type	Numeric
Duration	Duration	Numeric

Data availability The data that support the findings of this study are openly available on public repository: http://promise.site.uottawa.ca/SERepository/datasets-page.html.

Declarations

Conflict of interest The authors declare that they have no known conflict interest or personal relationships that could have appeared to influence the work reported in this paper."

References

1. Alqasrawi Y, Azzeh M, Elsheikh Y (2022) Locally weighted regression with different kernel smoothers for software effort estimation. Sci Comput Program 214:102744. https://doi.org/10.1016/J.SCICO.2021.102744
2. Tawosi V, Sarro F, Petrozziello A, Harman M (2022) Multi-objective software effort estimation: a replication study. IEEE Trans Softw Eng 48(8):3185–3205. https://doi.org/10.1109/TSE.2021.3083360
3. Minku LL (2019) A novel online supervised hyperparameter tuning procedure applied to cross-company software effort estimation. Empir Softw Eng 24(5):3153–3204. https://doi.org/10.1007/S10664-019-09686-W/TABLES/9

4. Xia T, Shu R, Shen X, Menzies T (2022) Sequential model optimization for software effort estimation. IEEE Trans Softw Eng 48(6):1994–2009. https://doi.org/10.1109/TSE.2020.3047072
5. Phannachitta P (2020) On an optimal analogy-based software effort estimation. Inf Softw Technol 125:106330. https://doi.org/10.1016/J.INFSOF.2020.106330
6. Azzeh M, Nassif AB (2017) Analyzing the relationship between project productivity and environment factors in the use case points method. J Softw: Evolut Process. https://doi.org/10.1002/smr.1882
7. Keung J, Kocaguneli E, Menzies T (2013) Finding conclusion stability for selecting the best effort predictor in software effort estimation. Autom Softw Eng 20(4):543–567. https://doi.org/10.1007/s10515-012-0108-5
8. Kocaguneli E, Menzies T, Keung JW (2012) On the value of ensemble effort estimation. IEEE Trans Software Eng 38(6):1403–1416. https://doi.org/10.1109/TSE.2011.111
9. Kassaymeh S, Alweshah M, Al-Betar MA, Hammouri AI, M. A. Al-Ma'aitah, (2023) Software effort estimation modeling and fully connected artificial neural network optimization using soft computing techniques. Cluster Comput. https://doi.org/10.1007/S10586-023-03979-Y/METRICS
10. Pal N, Yadav MP, Yadav DK (2023) Appropriate number of analogues in analogy based software effort estimation using quality datasets. Cluster Comput. https://doi.org/10.1007/S10586-023-03967-2/METRICS
11. Pantoni RP, Mossin EA, Brandão D (2008) Task effort fuzzy estimator for software development. INFOCOMP J Comput Sci 7(2):84–89
12. Minku LL (2021) Multi-stream online transfer learning for software effort estimation: Is it necessary?. In: Proceedings of the 17th international conference on predictive models and data analytics in software engineeringhttps://doi.org/10.1145/3475960.3475988.
13. Azzeh M, Nassif AB (2013) Fuzzy Model Tree for early effort estimation. In: 2013 12th international conference on machine learning and applications. https://doi.org/10.1109/ICMLA.2013.115.
14. Kocaguneli E, Menzies T, Keung J, Cok D, Madachy R (2013) Active Learning and effort estimation: finding the essential content of software effort estimation data. IEEE Trans Softw Eng 39(8):1040–1053. https://doi.org/10.1109/TSE.2012.88
15. Rao KE, Rao GA, Rao E (2020) "Retraction Note: ensemble learning with recursive feature elimination integrated software effort estimation: a novel approach. Evolut Intell 14:151–162. https://doi.org/10.1007/s12065-020-00360-5
16. Kocaguneli E, Menzies T, Bener AB, Keung JW (2012) Exploiting the essential assumptions of analogy-based effort estimation. IEEE Trans Software Eng 38(2):425–438. https://doi.org/10.1109/TSE.2011.27
17. de Morais RA (2021) Deep learning based models for software effort estimation using story points in agile environments. https://doi.org/10.7939/R3-JCF5-8X08
18. Phan H, Jannesari A (2022) Story point effort estimation by text level graph neural network. Accessed 31 Mar 2023. [Online]. Available: https://arxiv.org/abs/2203.03062v2
19. Kassem H, Mahar K, Saad AA (2023) Story point estimation using issue reports with deep attention neural network. e-Inf Softw Eng J 17(1):230104. https://doi.org/10.37190/e-Inf230104
20. Marapelli B, Carie A, Islam SM ((2020) RNN-CNN model: A bi-directional long short-term memory deep learning network for story point estimation. In: CITISIA 2020 5th international conference on innovative technologies in intelligent systems and industrial applications. https://doi.org/10.1109/CITISIA50690.2020.9371770.
21. Choetkiertikul M, Dam HK, Tran T, Pham T, Ghose A, Menzies T (2019) A deep learning model for estimating story points. IEEE Trans Softw Eng 45(7):637–656. https://doi.org/10.1109/TSE.2018.2792473
22. Zadeh LA (1997) Toward a theory of fuzzy information granulation and its centrality in human reasoning and fuzzy logic. Fuzzy Sets Syst 90(2):111–127. https://doi.org/10.1016/S0165-0114(97)00077-8
23. Bezdek JC, Ehrlich R, Full W (1984) FCM: the fuzzy c-means clustering algorithm. Comput Geosci 10(2–3):191–203. https://doi.org/10.1016/0098-3004(84)90020-7
24. Liu M, Shi J, Li Z, Li C, Zhu J, Liu S (2017) Towards better analysis of deep convolutional neural networks. IEEE Trans Vis Comput Graph 23(1):91–100. https://doi.org/10.1109/TVCG.2016.2598831
25. Khan A, Sohail A, Zahoora U, Qureshi AS (2020) A survey of the recent architectures of deep convolutional neural networks. Artif Intell Rev 53(8):5455–5516. https://doi.org/10.1007/s10462-020-09825-6
26. Kumar PS, Behera HS, Kumari A, Nayak J, Naik B (2020) Advancement from neural networks to deep learning in software effort estimation: perspective of two decades. Comput Sci Rev 38:100288
27. Kocaguneli E, Menzies T, Keung JW (2013) Kernel methods for software effort estimation: effects of different kernel functions and bandwidths on estimation accuracy. Empir Softw Eng 18(1):1–24. https://doi.org/10.1007/s10664-011-9189-1
28 Azzeh M, Nassi AB (2015) Analogy-based effort estimation: a new method to discover set of analogies from dataset characteristics. IET Softw. https://doi.org/10.1049/iet-sen.2013.0165
29 Jose Thiago JTH, Oliveira ALI (2021) Ensemble effort estimation using dynamic selection. J Syst Softw 175:110904. https://doi.org/10.1016/J.JSS.2021.110904
30. Nassif AB, Azzeh M, Capretz LF, Ho D (2013) A comparison between decision trees and decision tree forest models for software development effort estimation. In: 2013 Third international conference on communications and information technology (ICCIT). https://doi.org/10.1109/ICCITechnology.2013.6579553.
31. Zhang J, Chen L (2019) Clustering-based undersampling with random over sampling examples and support vector machine for imbalanced classification of breast cancer diagnosis. Comput Assist Surg 24(sup2):62–72. https://doi.org/10.1080/24699322.2019.1649074
32. Azzeh M, Nassif AB (2016) A hybrid model for estimating software project effort from use case points. Appl Soft Comput J 49:981–989. https://doi.org/10.1016/j.asoc.2016.05.008
33. Sarno R, Sidabutar J (2015) Improving the accuracy of COCOMO's effort estimation based on neural networks and fuzzy logic model. In: 2015 International Conference on Information & Communication Technology and Systems (ICTS) Accessed 07 Sep 2023. [Online]. Available: https://ieeexplore.ieee.org/abstract/document/7379898/
34. Menzies T, Chen Z, Hihn J, Lum K (2006) Selecting best practices for effort estimation. IEEE Trans Softw Eng 32(11):883–895. https://doi.org/10.1109/TSE.2006.114
35 Azzeh M, Nassif AB (2018) Project productivity evaluation in early software effort estimation. J Softw: Evolut Process. https://doi.org/10.1002/smr.2110
36. Kaushik A, Kaur P, Choudhary N (2022) Stacking regularization in analogy-based software effort estimation. Soft comput 26(3):1197–1216. https://doi.org/10.1007/S00500-021-06564-W/FIGURES/9
37. Jørgensen M, Halkjelsvik T (2020) Sequence effects in the estimation of software development effort. J Syst Softw 159:110448. https://doi.org/10.1016/j.jss.2019.110448

38 Azzeh M, Nassif AB, Minku LL (2015) An empirical evaluation of ensemble adjustment methods for analogy-based effort estimation. J Syst Softw. https://doi.org/10.1016/j.jss.2015.01.028
39 Minku LL, Yao X (2013) Ensembles and locality: insight on improving software effort estimation. Inf Softw Technol 55(8):1512–1528. https://doi.org/10.1016/j.infsof.2012.09.012
40. Gallego JJC, Rodríguez D, Sicilia MÁ, Rubio MG, Crespo AG (2007) Software project effort estimation based on multiple parametric models generated through data clustering. J Comput Sci Technol 22(3):371–378. https://doi.org/10.1007/s11390-007-9043-5
41 Azzeh M, Nassif AB, Martín CL (2021) Empirical analysis on productivity prediction and locality for use case points method. Softw Qual J 29(2):309–336. https://doi.org/10.1007/s11219-021-09547-0
42 Kaur P, Singh R (2015) A proposed framework for software effort estimation using the combinational approach of fuzzy logic and neural networks. Int J Hybrid Inf Technol 8(10):73–80. https://doi.org/10.14257/ijhit.2015.8.10.07
43. Amazal FA, Idri A (2020) Estimating software development effort using fuzzy clustering-based analogy. J Softw: Evolut Process. https://doi.org/10.1002/smr.2324
44 Nassif AB, Azzeh M, Idri A, Abran A (2019) Software development effort estimation using regression fuzzy models. Comput Intell Neurosci. https://doi.org/10.1155/2019/8367214
45. Azzeh M, Neagu D, Cowling P (2008) Adjusting analogy software effort estimation based on fuzzy logic. In: ICSOFT 2008-Proceedings of the 3rd international conference on software and data technologies
46. Xie XL, Beni G (1991) A validity measure for fuzzy clustering. IEEE Trans Pattern Anal Mach Intell 13(08):841–847. https://doi.org/10.1109/34.85677
47. Menzies B, Caglayan B, Kocaguneli E, Krall J, Peters F. The PROMISE repository of empirical software engineering. PROMISE. https://scholar.google.com/scholar?hl=en&as_sdt=0%2C5&q=The+PROMISE+Repository+of+empirical+software+engineering+data+&btnG= (Accessed 27 Dec 2020)
48. Azzeh M, Nassif AB, Elsheikh Y, Angelis L (2022) On the value of project productivity for early effort estimation. Sci Comput Program 219:102819. https://doi.org/10.1016/J.SCICO.2022.102819
49. Kocaguneli E, Menzies T (2013) Software effort models should be assessed via leave-one-out validation. J Syst Softw 86(7):1879–1890. https://doi.org/10.1016/j.jss.2013.02.053

Publisher's Note Springer Nature remains neutral with regard to jurisdictional claims in published maps and institutional affiliations.

Springer Nature or its licensor (e.g. a society or other partner) holds exclusive rights to this article under a publishing agreement with the author(s) or other rightsholder(s); author self-archiving of the accepted manuscript version of this article is solely governed by the terms of such publishing agreement and applicable law.